

**Midterm Report: Exploring Columnar Execution in DuckDB through a Sales Analytics
Application**
Yunhui Dong

1. Database System Overview and Architecture

For my project, I have selected DuckDB as my target database. DuckDB is an analytical database designed for OLAP workloads. The guide provided for the class also refers to it as a columnar and analytical database. This is a good choice for exploring how internal database design affects analytical applications.

My primary reason for choosing DuckDB is its columnar data storage and vectorized execution. In my initial project proposal, I planned on relating both of these internal features to a simple sales analytics application written in Python. The idea is to relate how an analytical application's queries, such as grouping and date filtering, relate to DuckDB's internal execution behavior.

When compared to a row-based database like MySQL, DuckDB is designed for applications where queries need to scan through specific columns, filter data, and calculate sums and averages for large amounts of data. The guide for our project refers to DuckDB as scanning only through columns it needs to scan and executes in cache-friendly batches.

2. Focus Areas: Columnar Storage and Vectorized Query Execution

This project examines two internal features of DuckDB, which are columnar storage and vectorized execution.

First, columnar storage is important in the sense that most analytical queries only require a few attributes. For instance, if the query is to find the total sales made in different cities, it would involve the city and sales attributes. It would not require other attributes, such as gender, method of payment, or rating. This is the major reason columnar storage is useful, especially when it comes to reporting and business analytics.

Second, DuckDB uses vectorized query execution. It executes queries in batches rather than a single row at a time. This is especially beneficial when doing scans, filters, projections, and grouped aggregations. As my application is primarily concerned with these operations, it is

clear from DuckDB how internal execution strategies facilitate the working of the application itself.

3. Preliminary Application Design

The application for the project is a sales analytics problem, and it's implemented by using Python and DuckDB. The aim of this project is not necessarily to build a fancy interface, but it's a simple analytics workload that demonstrates how DuckDB works internally, along with representative SQL queries. It's centered on a database and how the internals of the database relate to what the application does.

For the checkpoint, a supermarket sales CSV file is used, and it's imported into DuckDB as a table called `supermarket_sales`. The table contains columns such as invoice ID, branch, city, customer type, gender, product line, quantity, sales, date, payment method, gross income, and rating.

The main queries currently implemented are:

- total sales by city,
- average sales by product line,
- total sales in January 2019,
- transaction count and total sales by payment method,
- total sales by branch and product line.

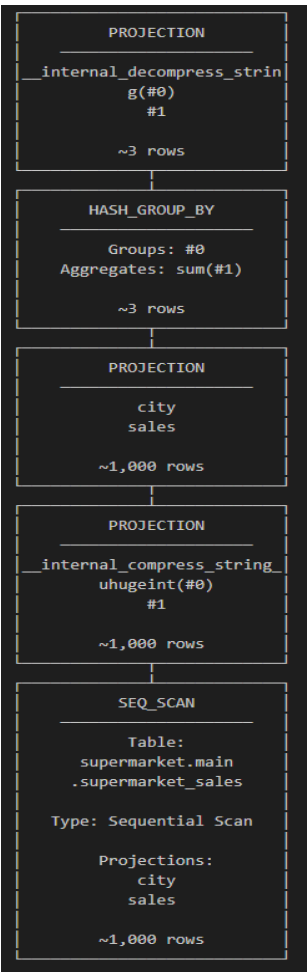
These queries make the problem concrete and provide examples of grouped aggregation and filtering, which is a key focus of this project.

4. Preliminary Mapping Between Internals and Application Behavior

One key part of this project is to explain what the app is doing, what the database is doing internally, and why the behavior is important. The current implementation already provides several examples of this mapping.

Regarding the query used to calculate the total sales by city, the app generates a business summary by locations. Internally, the DuckDB database uses a grouped aggregation and works mainly with the city and sales columns used in the query. In the current output of the

EXPLAIN function, a HASH_GROUP_BY operator is used, which supports the idea that the query is using the column-oriented analytical execution style.



The average sales by product line query has a similar structure. At the application level, this query is similar to the previous one because it uses the average sales query. At the database level, only the grouping column and the total sales column are used.

The query on the dates in January of 2019 is another common analytical query. At the application level, the query is asking for the total sales in a specific time frame. At the database level, only the date column and the sales column are used.

			product_line	avg_sales			
			0	Home and lifestyle	336.636956	jan_sales	
			1	Sports and travel	332.065220		
			2	Health and beauty	323.643020		
			3	Food and beverages	322.671517		
			4	Electronic accessories	319.632538		
			5	Fashion accessories	305.089298		
city	total_sales					0	
0	Naypyitaw	110568.7065					
1	Yangon	106200.3705					
2	Mandalay	106197.6720				116291.868	

These three queries show that the project is not based on a single query on a toy database. It has a small but meaningful analytical workload that could be used to analyze the internal behavior of DuckDB.

5. Progress Update and Challenges

At this point, the implementation checkpoint requirements are satisfied. The DuckDB implementation is already set up in Python, the CSV data is already loaded into an initial schema, and several of the analytical queries are already functioning. An initial EXPLAIN plan for the main query used in the application's grouping aggregation query has also been created, which demonstrates the relationship between running the application and the database.

The remaining problem with the implementation so far is that the current data set being used is relatively small, so performance differences might not be readily apparent. For the final implementation, an EXPLAIN ANALYZE result will be added, as well as a possibly larger synthetic data set to test the performance of the DuckDB execution behavior. Another problem with the implementation so far is that vectorized execution needs to be demonstrated, not just through an output of a query plan, but also through runtime performance.

6. Conclusion

Currently, the project has a good foundation in place. DuckDB has been set up correctly, the sales dataset has been imported, and several analytical queries have been implemented. The current results already validate the initial direction of the project, and analytical application behaviors such as group aggregation and date filtering can be supported by the column storage and vectorized execution paradigm provided by DuckDB. This provides a good basis for the final implementation and report.

References

Raasveldt, Mark, and Hannes Mühleisen. "DuckDB." *Proceedings of the 2019 International Conference on Management of Data*, 2019.

DuckDB Documentation. "Why DuckDB."